



Golf Scorekeeper App

Architecture Strategy, Contract Plan & Code Refactor Recommendations

Prepared from design session · April 2026

Confidential working document

Background & Strategic Context

The app has grown to a level of complexity — 7 side games, press hierarchies, subset pickers, handicap modes, multi-player match formats — where informal "build as you go" development stops scaling. Two parallel investments are now warranted:

1. **Specification Contracts** — written rules each game and subsystem must conform to, enabling parallel developer work and making bugs easier to identify and fix
2. **Code Architecture Refactor** — breaking large files into independently maintainable modules, reducing the largest file from 1,627 lines to ~250

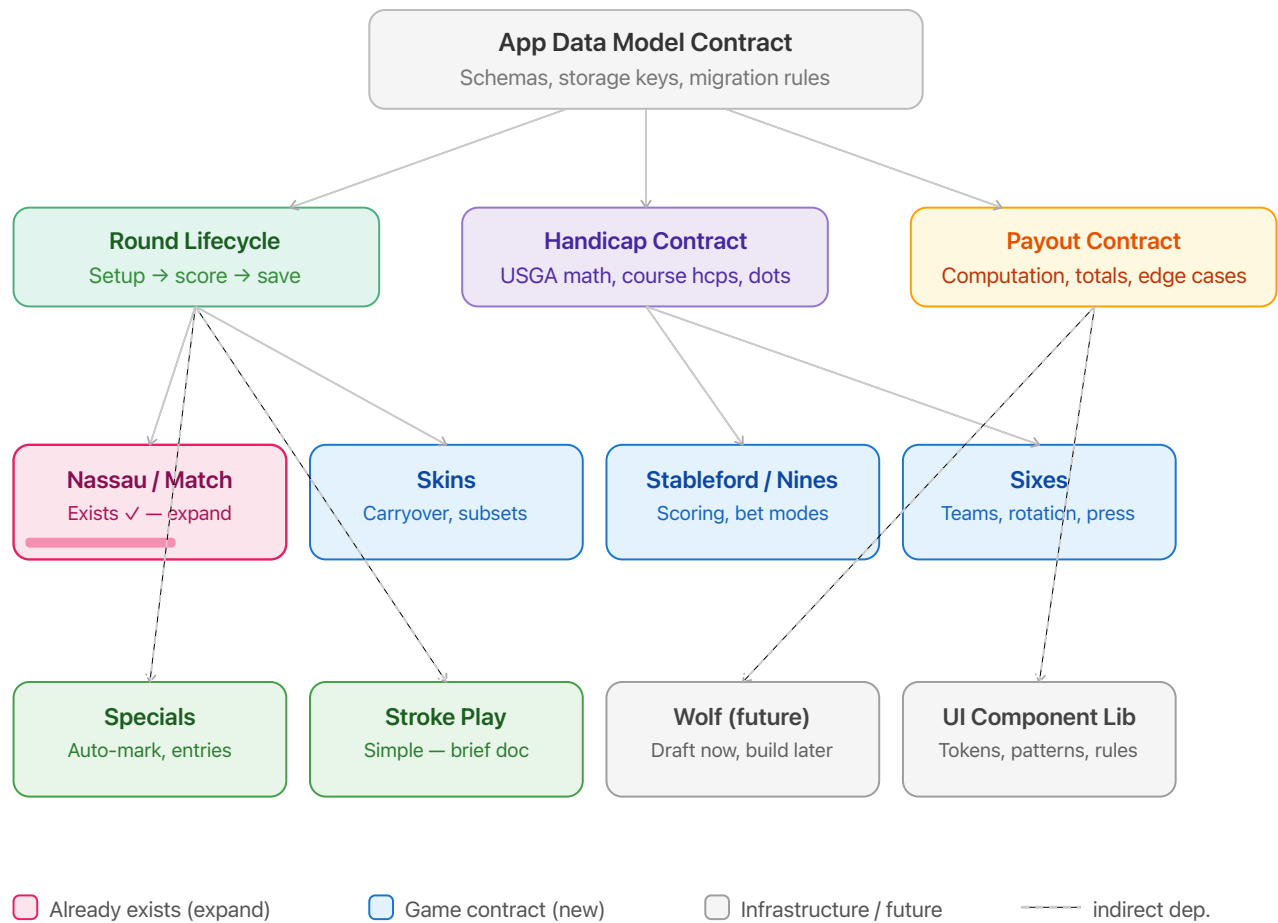
Recommended sequence:

Contracts → Code split → Fix & test. Writing contracts first defines what "correct" means before any bugs are fixed. The code split then makes fixing easier by isolating each game into its own file.

Part 1 — Specification Contracts

The existing `Match_Nassau_Contract.md` is the right model. The approach is **one contract per logical domain**, not one per game. Nine contracts are recommended:

CONTRACT ARCHITECTURE — DEPENDENCY MAP



The 9 Contracts — Detail

CONTRACT 1**App Data Model** New

All localStorage keys and schema shapes. The activeRound blob fields (authoritative list). History record schema. Migration rules. makeId() conventions. Foundation for all other contracts.

CONTRACT 2**Round Lifecycle** New

State machine: setup → scoring → results → saved. What triggers each transition. What buildPayoutArgs() must receive and return. Error handling and recovery for corrupt active round.

CONTRACT 3**Handicap** New

USGA math: course handicap formula. scoreForMode behavior for gross/net/netofflow. Stroke allocation rules per hole. groupCourseHandicaps API. Handicap dot display rules (open item — decide here).

CONTRACT 4**Payout** New

computePayouts() inputs/outputs. Per-game breakdown format. How ties are resolved per game. Totals aggregation rules.

CONTRACT 5**Nassau / Match** Exists ✓

Already documented. Expand to add: team match rules, per-segment auto-press triggering spec, tiebreak modes (second/half), netofflow scoring definition.

CONTRACT 6**Skins** New

Carryover rules. Subset player behavior. Tied hole vs. carryover vs. no-carryover logic. Display must respect skinsPlayers subset (open item — fixed by this contract).

CONTRACT 7**Stableford / Nines** New

Combined — both are point-based. Stableford points table (default + custom). stabBetMode: nassau vs perpoint — full payout spec. Nines: point assignment (9/5/3/1/0), blitz rule, 3-player constraint.

CONTRACT 8**Sixes** New

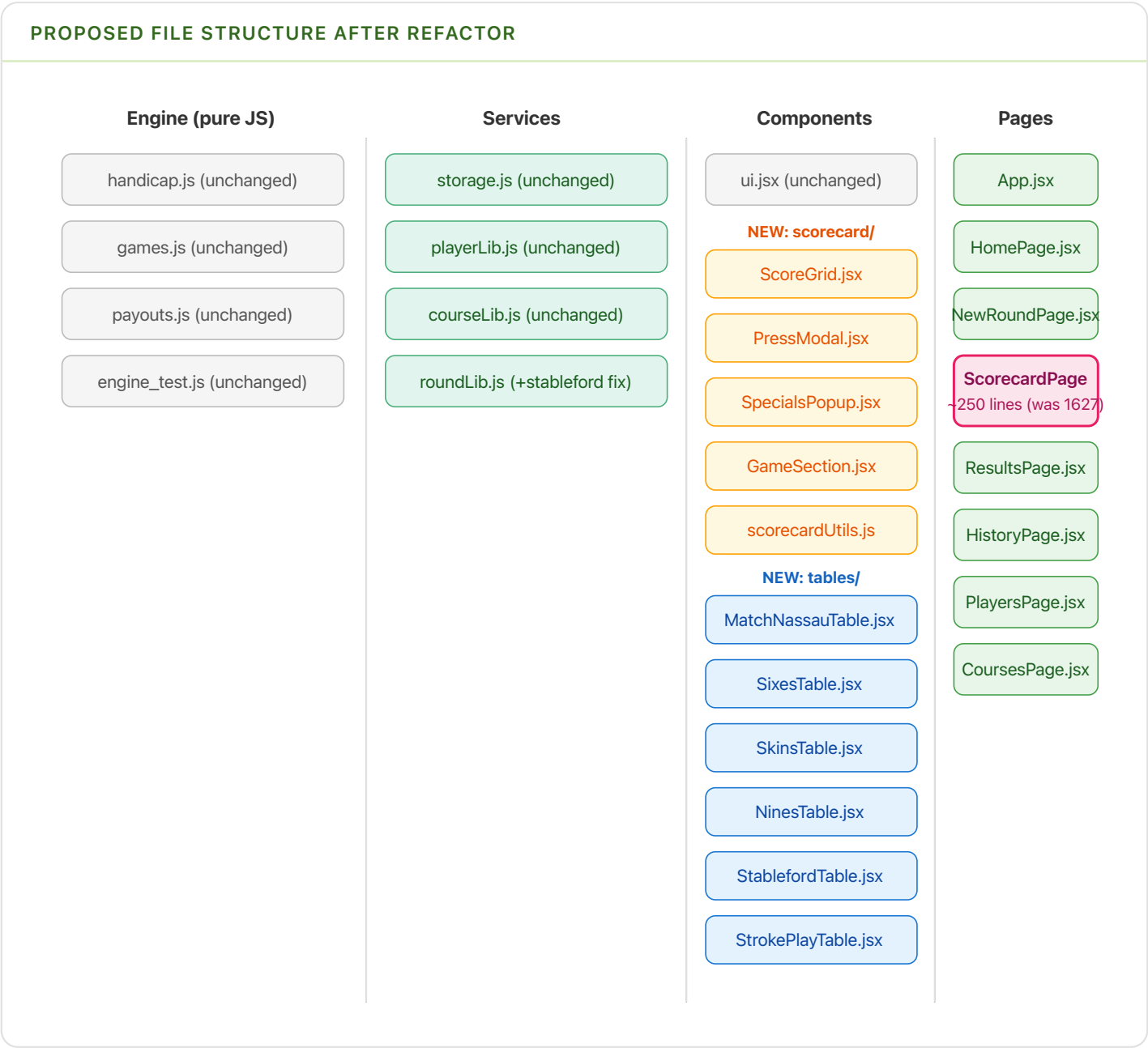
Team rotation rules across 6-hole segments. Press behavior (currently an open item — spec it here). Scoring and payout. Resolves the open Sixes press wiring bug.

CONTRACT 9**UI Component & Style** New

Design tokens (G, GA, RED, AMB, etc.). Component API for Btn, Inp, Sel, Toggle, BetInput, Card. Layout patterns. Rules for new game table components (colors, structure, shared shells).

Part 2 — Code Architecture Refactor

The 1,627-line ScorecardPage.jsx is the biggest pain point, but there are broader gains across several files. The goal is to make each game independently editable — ideally by a different developer — without side-effects on other games.



ScorecardPage.jsx Breakdown — Line Count by Extraction

Extract to	Est. Lines	What moves there
scorecard/ScoreGrid.jsx	~300	Hole-by-hole grid, PopDots, SpecialsPopup, long-press logic
scorecard/PressModal.jsx	~120	PressModal, SinglePressChip, SegmentChipColumns
scorecard/GameSection.jsx	~30	GameSection, HalfLabel, TableDivider, ColNote shared shells
scorecard/scorecardUtils.js	~50	restoreAutoWhen, GameTable, PlayerChips, buildLeadState, fmtLead, design tokens (M, MP, N, S...)
tables/MatchNassauTable.jsx	~380	Most complex — keeps all its internal helpers
tables/SixesTable.jsx	~185	Sixes rendering and team logic
tables/SkinsTable.jsx	~75	Skins hole grid
tables/NinesTable.jsx	~65	Nines point table
tables/StablefordTable.jsx	~55	Stableford hole grid
tables/StrokePlayTable.jsx	~75	Stroke play grid
ScorecardPage.jsx (residual)	~200	State management, renderTotalsCards, main Scorecard orchestrator, ScorecardPage export

Other Files to Refactor

File (current size)	Extract	Benefit
HistoryPage.jsx (1,084 lines)	HistoryRoundDetail.jsx (~400 lines)	Makes "render matches properly" open item easy to scope independently
CoursesPage.jsx (1,036 lines)	CourseCard.jsx, CourseMergeModal.jsx	Both are independently testable; currently buried
NewRoundPage.jsx (974 lines)	MatchCard.jsx, GameConfig.jsx	Unlocks parallel work on game setup UI

Part 3 — Recommended Sequence

Key principle:

Contracts → Code split → Fix & test. Don't fix bugs in the 1,627-line file then split — do careful work in hard-to-

navigate files. Split first (mechanical, low-risk), then fix in the smaller files.

Two parallel tracks

Track A — Contracts + architecture (do first, unlocks everything)

- 1Write contracts for already-working games
- Nassau/Match (expand existing), Skins, Stableford, Stroke Play. No code changes.
- 2Do the code split
- Pure extract-and-re-export. Zero logic changes. Start with MatchNassauTable — most complex, highest value to isolate.
- 3Fix and test in the smaller files
- Each game table is now independently editable. Bugs are easier to scope, verify, and hand off.

Track B — Rules decisions for broken/incomplete games (can run in parallel)

- ADecide Sixes press behavior
- Write Sixes contract → fix Sixes press wiring in the extracted SixesTable.jsx
- BDecide handicap dot display strategy
- Write that section of the Handicap contract → implement in ScoreGrid.jsx
- CResolve remaining open items
- stableford_players in roundLib, SkinsTable subset display, matches[] in HistoryPage — each now has a clear home file

High-Priority Open Items (pre-existing, to address in parallel)

Item	File (after split)	Blocking?
Add stableford_players to roundLib converters	roundLib.js	Yes — data loss on save/reload
Sixes press wiring in payouts.js	tables/SixesTable.jsx + payouts.js	Yes — Sixes payouts wrong
Render matches[] in HistoryPage detail	HistoryRoundDetail.jsx	No — cosmetic
SkinsTable subset filter in display	tables/SkinsTable.jsx	No — engine already correct
Nassau/Match testing pass	tables/MatchNassauTable.jsx	Yes — verify before release

Golf Scorekeeper — Architecture & Contract Plan — April 2026 — 100% Complete